Компиляторы. ANF

Косарев Дмитрий

13 сентября 2025 г.

Дата сборки: 13 сентября 2025 г.



ANF — упрощённое представление программ

Мы будем упрощать выражения, чтобы аргументами функций были «атомарные» (immediate) выражения (константы, имена и т.д.)

```
type expr =
| Num of int
| Id of string
| Plus of expr * expr
| Minus of expr * expr
| Let of string * expr * expr
```

$\Longrightarrow$

```
type immexpr =
| ImmNum of int
| ImmId of string

type cexpr =
| CPlus of immexpr * immexpr

type aexpr =
| ALet of string * cexpr * aexpr
```

Впервые упомянуты в 1993 в статье “The Essence of Compiling with Continuations” [1]

ANF: Примеры и уточнения

1. (5 + 4) − 2
2. let x = 5 in x

```
type immexpr =
| ImmNum of int
| ImmId of string

type cexpr =
| CPlus of immexpr * immexpr

type aexpr =
| ALet of string * cexpr * aexpr
```

ANF: Примеры и уточнения

1. (5 + 4) − 2
   let v1 = 5 + 4 in v1 − 2
2. let x = 5 in x

```
type immexpr =
| ImmNum of int
| ImmId of string

type cexpr =
| CPlus of immexpr * immexpr

type aexpr =
| ALet of string * cexpr * aexpr
```

ANF представление — новые конструкторы

```
type cexpr =
| CPlus of immexpr * immexpr
| CImmExpr of immexpr

type aexpr =
| ALet of string * cexpr * aexpr
| ACExpr of cexpr
```

ANF представление — сложный случай

Plus (..., ...) . Не совсем понятно, что делать с результатами левого и правого слагаемого, так как там могут появиться много let. Мэтчиться по ним не хочется. Поэтому...

```
let rec anf (expr_with_hole : immexpr -> aexpr) = function
  | Num n -> expr_with_hole (ImmNum n)
  | Id x -> expr_with_hole (ImmId x)
  (* | ... *)
```

ANF представление — сложный случай

Plus (..., ...) . Не совсем понятно, что делать с результатами левого и правого слагаемого, так как там могут появиться много let. Мэтчиться по ним не хочется. Поэтому...

```
let rec anf (expr_with_hole : immexpr -> aexpr) = function
  | Num n -> expr_with_hole (ImmNum n)
  | Id x -> expr_with_hole (ImmId x)
  (* | ... *)
  | Plus(left, right) ->
      anf left (fun limm ->
        anf right (fun rimm ->
          (* bad *) CPlus (limm, rimm)
```

ANF представление — сложный случай

Plus (..., ...) . Не совсем понятно, что делать с результатами левого и правого слагаемого, так как там могут появиться много let. Мэтчиться по ним не хочется. Поэтому...

```
let rec anf (expr_with_hole : immexpr -> aexpr) = function
  | Num n -> expr_with_hole (ImmNum n)
  | Id x -> expr_with_hole (ImmId x)
  (* | ... *)
  | Plus(left, right) ->
      anf left (fun limm ->
        anf right (fun rimm ->

          (* OK *) ALet("result_of_plus",
                        CPlus(limm, rimm),
                        expr_with_hole (ImmId "result_of_plus"))
      ))
```

Заключение про ANF I

- ANF пригодится, когда дойдете до LLVM
- Можно написать много разных реализаций, самое нетривиальные места — `let` и `if`
- Часто будут появляться ненужные переменные, например, `(let x1 = x3 in x1)`. Можно пытаться такое не порождать, а можно упрощать отдельным проходом
- В контексте miniML могут появляться каверзные случаи, типа `(let temp1 = (+) x1 in temp1 x2)` вместо `(x1 + x2)`. Используя информацию об уже объявленных функциях, нужно не создавать такие ANF

  `val (+): int` $\xrightarrow{\text{pure}}$ `int` $\xrightarrow{\text{pure}}$ `int`

Заключение про ANF II

- У LLVM и ассемблера RISC-V есть специальные инструкции для сравнения и перехода. Поэтому программы получатся проще, если «инлайнить» `let x = A < B in if x then ...` в `if A<B then ...`
- Вывод: если сделать проверку типов (арность, стандартная ли это функция печати/сложения или пользовательская) до ANF, то будут менее «мусорные» ANF
- **Недостаток**: иногда рядом с if-then-else код копируется и размер программы «взрывается»
- **Недостаток**: не замкнута, относительное β-редукции, inlining затруднён

Ссылки & Acknowledgements

- Cormac Flanagan и др. «The essence of compiling with continuations». В: 1993. URL: https://doi.org/10.1145/155090.155113.
- Joe Gibbs Politz. *Principles of Compiler Design: ANF*. 2016. URL: https://www.cs.swarthmore.edu/~jpolitz/cs75/s16/n_anf-tutorial.html.
- Xavier Leroy. *Part V: functional intermediate representations*. URL: https://xavierleroy.org/mpri/2-4/fir.pdf.
- William J. Bowman. «A Low-Level Look at A-Normal Form». В: *Proc. ACM Program. Lang.* OOPSLA2 (2024). URL: https://doi.org/10.1145/3689717.